



University of Central Punjab
Faculty of Information Technology & Computer Sciences

Data Structures, Spring 2026

Assignment 3

Total Marks: 40		Deadline: 22 – 05 - 2026		Section:	
Name:			Reg. #:		
CLO	CLO Statement		Taxonomy Level	PLO	
2	Implement various data structures and apply them in simple applications.		C2	4	

Instructions: (5 marks will be deducted for not following the instructions)

- You have to submit Assignment 3 via Portal. Do individually.
- Quiz will be taken on the basis of assignment.
- In case of plagiarism, **straight ZERO** will be awarded.
- Attempt all questions in sequence. **Attach this title page as a front page of assignment.**
- Assignments will not be accepted on email and late submissions of assignments are strictly prohibited.
- Attach output screen shot of each question.
- Submit your code with screenshots of the result running over terminal

Answers Clarity: Justify your answer where needed.



University of Central Punjab

Faculty of Information Technology & Computer Sciences

Question # 1: Design an undo/redo system for a text editor where each action (insert/delete) is stored as a node in a doubly linked list. The system should support:

- **Undo:** Revert to the previous state.
- **Redo:** Reapply a previously undone action.
- **Limit History:** Keep only the last n actions (memory-efficient).

Requirements:

1. Use a doubly linked list with head and tail pointers.
2. Implement:
 - `add_action(action)` – Adds a new action to the list.
 - `undo()` – Moves backward in history.
 - `redo()` – Moves forward in history.
 - `limit_history(n)` – Truncates the list to keep only n recent actions.
3. Each node should store:
 - `action_type` ("insert" or "delete")
 - `text` (affected characters)
 - `position` (index in the document)

Sample Output:

```
===== TEXT EDITOR UNDO/REDO SYSTEM =====
Action Performed: Insert "H" at position 0
Current Document: H
Action Performed: Insert "e" at position 1
Current Document: He
Action Performed: Insert "l" at position 2
Current Document: Hel
Action Performed: Insert "l" at position 3
Current Document: Hell
Action Performed: Insert "o" at position 4
Current Document: Hello
Action Performed: Delete "o" from position 4
Current Document: Hell

----- ACTION HISTORY -----
1. Insert "H" at position 0
2. Insert "e" at position 1
3. Insert "l" at position 2
4. Insert "l" at position 3
5. Insert "o" at position 4
6. Delete "o" from position 4
-----

Undo Operation Performed
Reverted: Delete "o" from position 4
Current Document: Hello
Undo Operation Performed
```



University of Central Punjab

Faculty of Information Technology & Computer Sciences

```
Reverted: Insert "o" at position 4
Current Document: Hell
Redo Operation Performed
Reapplied: Insert "o" at position 4
Current Document: Hello

Adding New Action:
Insert "!" at position 5
Current Document: Hello!

Redo Stack Cleared (new action added after undo)

----- UPDATED HISTORY -----
1. Insert "H" at position 0
2. Insert "e" at position 1
3. Insert "l" at position 2
4. Insert "l" at position 3
5. Insert "o" at position 4
6. Insert "!" at position 5
-----

Applying History Limit: 4 actions

Oldest actions removed successfully.

----- HISTORY AFTER TRUNCATION -----
1. Insert "l" at position 2
2. Insert "l" at position 3
3. Insert "o" at position 4
4. Insert "!" at position 5
-----

Tail Node:
Action Type: insert
Text: !
Position: 5

Current Document State: Hello!

===== PROGRAM TERMINATED =====
```

Question # 2: Design a recursive program to help a robot escape from a maze.

Maze Symbols

- # → Wall
- . → Open Path
- D → Destination

Requirements

1. The robot starts from position (0,0).
2. Use recursion only to explore the maze.
3. The robot can move:
 - Up



University of Central Punjab

Faculty of Information Technology & Computer Sciences

- Down
 - Left
 - Right
4. Find a valid path from source to destination.
 5. Print the maze with the successful path marked using *.
 6. Count and display the total number of recursive calls.
 7. Mark visited cells to avoid infinite recursion.

Restrictions

- No loops allowed in path traversal logic.
- No STL containers.
- Use recursion only.

Sample Input Maze:

```
..#..
#.#.#
.....
.###.
...D.
```

Sample Output:

```
* * # ..
# * # .#
. * * *
.###*
...D*
```

Destination Found!

Path:

$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4) \rightarrow (4,3)$

Total Recursive Calls = 31



University of Central Punjab

Faculty of Information Technology & Computer Sciences